Repair Ingredients Are All You Need: Improving Large Language Model-Based Program Repair via Repair Ingredients Search

Jiayi Zhang*
Nanyang Technological University
Singapore, Singapore
jzhang150@e.ntu.edu.sg

Kai Huang*
Technical University of Munich
München, Germany
kai-kevin.huang@tum.de

Jian Zhang†
Nanyang Technological University
Singapore, Singapore
zhangj3353@gmail.com

Yang Liu
Nanyang Technological University
Singapore, Singapore
yangliu@ntu.edu.sg

Chunyang Chen
Technical University of Munich
München, Germany
chun-yang.chen@tum.de

Abstract

Automated Program Repair (APR) techniques aim to automatically fix buggy programs. Among these, Large Language Model-based (LLM-based) approaches have shown great promise. Recent advances demonstrate that directly leveraging LLMs can achieve leading results. However, these techniques remain suboptimal in generating contextually relevant and accurate patches, as they often overlook repair ingredients crucial for practical program repair. In this paper, we propose ReinFix, a novel framework that enables LLMs to autonomously search for repair ingredients throughout both the reasoning and solution phases of bug fixing. In the reasoning phase, ReinFix integrates static analysis tools to retrieve internal ingredients, such as variable definitions, to assist the LLM in root cause analysis when it encounters difficulty understanding the context. During the solution phase, when the LLM lacks experience in fixing specific bugs, ReinFix searches for external ingredients from historical bug fixes with similar bug patterns, leveraging both the buggy code and its root cause to guide the LLM in identifying appropriate repair actions, thereby increasing the likelihood of generating correct patches. Evaluations on two popular benchmarks (Defects4J V1.2 and V2.0) demonstrate the effectiveness of our approach over SOTA baselines. Notably, ReinFix fixes 146 bugs, which is 32 more than the baselines on Defects4J V1.2. On Defects4J V2.0, ReinFix fixes 38 more bugs than the SOTA. Importantly, when evaluating on the recent benchmarks that are free of data leakage risk, ReinFix also maintains the best performance.

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

*Joint first authors, contributed equally with author ordering decided by *Nigiri*.
†Corresponding author.

ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3764534

Keywords

Automated Program Repair, Large Language Model, Repair Ingredients, Agent-based Systems

ACM Reference Format:
Jiayi Zhang, Kai Huang, Jian Zhang, Yang Liu, and Chunyang Chen. 2026. Repair Ingredients Are All You Need: Improving Large Language Model-Based Program Repair via Repair Ingredients Search. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3764534

1 Introduction

Automated Program Repair (APR) aims to automatically fix software bugs [6, 9, 23, 33, 58]. Traditional APR techniques, including search-based [22], constraint-based [34], and template-based [28] methods, have made considerable strides in addressing common bug patterns. Template-based approaches, for example, have advanced in addressing common bugs but are restricted by the repair patterns to solve diverse bug types and complex repair scenarios [49]. To overcome these limitations, learning-based methods, particularly Neural Machine Translation (NMT)-based APR approaches [4, 15, 25, 26, 29, 31, 32, 41, 53, 57, 61, 62], frame bug fixing as a translation problem and learn the bug-fix domain knowledge from bug-fix pairs (BFPs) [41], where buggy code is "translated" into correct code using a model trained on BFPs.

Recently, the advent of Large Language Models (LLMs) has sparked a new wave of advancements in APR. Initially, many studies attempted to improve APR performance by fine-tuning LLMs, following similar strategies to NMT-based APR approaches [8, 13, 44], which remain constrained by limitations in training data. Therefore, more recent LLM-based approaches [2, 12, 19, 43, 45, 47–50, 54, 60] leverage the vast amount of knowledge encoded in pre-trained models, such as ChatGPT and Codex, which have been trained on extensive datasets of open-source code. These models demonstrate impressive performance on APR tasks by generating possible patches directly. One notable example is ChatRepair [49], a ChatGPT-based APR approach that combines patch generation with instant feedback, enabling conversational-style repair interactions. Similarly, ThinkRepair [54] uses Chain-of-Thought (CoT) prompting on ChatGPT to enhance bug reasoning during repair.

Despite their powerful bug-fixing capabilities, existing LLM-based APR works can struggle to produce contextually relevant and

accurate patches when essential repair ingredients are absent [10]. The repair ingredients is a concept built on the *redundancy assumption* [30], which falls into two categories: internal and external [51]. Internal repair ingredients include key contextual elements (e.g., variables) essential for understanding or addressing the bug, while external ingredients consist of repair behaviors (e.g., change actions) from prior fixes that may guide effective patch generation. In fact, research has demonstrated the importance of this redundancy assumption (i.e., repair ingredients) in traditional APR approaches [30]. On the one hand, redundant change actions from repair histories can often be distilled into fix patterns (or repair templates), guiding template-based APR methods by providing actionable repair workflows [18]. On the other hand, redundant code fragments have played a vital role in search-based APR systems as donor code for creating patches [22]. In the realm of LLM-based approaches, FitRepair [46] and RAP-Gen [42] are among the few representative works that utilize the redundancy assumption. FitRepair fine-tunes CodeT5 on the buggy project to learn internal code ingredients and prompts it to use these ingredients by searching relevant identifiers from the buggy project. Similarly, RAP-Gen retrieves external bug-fix pairs to augment the buggy input for the fine-tuning of CodeT5 as the patch generator.

However, existing strategies for using repair ingredients are primarily designed for small-scale LLMs and are not easily adaptable to advanced models. In addition, previous methods were straightforward in searching and utilizing repair ingredients, which may have resulted in imprecise ingredient search and patch generation. These issues stem from two key aspects: **1) Inaccurate Search for Repair Ingredients.** FitRepair collects internal code snippets exclusively from the buggy project for fine-tuning and searches for relevant identifiers through code similarity to enhance the prompted input, which may include irrelevant and redundant snippets. Similarly, RAP-Gen retrieves external bug-fix pairs based solely on the similarity of code, neglecting high-level contextual information about the bug (e.g., its causes), leading to a disconnect between bug analysis and appropriate fix action retrieval. **2) Limited Scalability of Repair Paradigms.** Both FitRepair and RAP-Gen utilize the fine-tuning paradigm based on CodeT5 to construct their repair workflows. Generalizing these approaches to mainstream LLMs (e.g., GPT-4) is challenging due to high cost of fine-tuning and it is not supported by some commercial models. Consequently, the integration of repair ingredients into LLMs to enhance repair capabilities remains underexplored, and the full potential of repair ingredients has yet to be unlocked in the era of LLMs.

Inspired by recent advances in LLM-based agents for software engineering tasks [2, 7, 24, 27], we recognize that these agents significantly enhance the versatility and expertise of LLMs by equipping them with the ability to perceive and utilize external resources and tools [27]. We believe that enabling agents to autonomously invoke tools to search for repair ingredients presents a promising avenue for improvement. To this end, we propose a novel framework, ReinFix (Reinforcement Ingredient Fixer), which employs LLM-based agents to integrate internal and external repair ingredients into the reasoning and solution phases of patch generation, respectively. ReinFix begins by reading the bug information, including buggy code and triggered test errors, and then initiates a planning step. In the reasoning phase, where project-specific identifiers (e.g., unique Java classes) may be unfamiliar to the LLM, ReinFix equips it with a toolkit capable of searching for definitions and details of those elements as internal repair ingredients via dependency analysis. This setup allows the LLM to invoke the tools by passing unknown identifiers as arguments. The additional information supports more accurate root-cause analysis, aiding in devising a valid solution for the bug. In the solution phase, ReinFix guides the LLM to analyze root causes from historical bug fixes, which are vectorized using an embedding model and stored in a vector database. Instead of solely considering code similarity as RAP-Gen did, we design a retrieval tool that searches for a external bug-fix pair with both similar bugs and root causes as repair patterns, enabling the LLM to generate more accurate and contextually relevant suggestions for the final generation of patches. To coordinate these processes, we implement our approach on the basis of the Reasoning and Acting framework [52], empowering the LLM to autonomously plan, analyze, and invoke the appropriate repair ingredients search tools as needed. This means that all tools are provided as optional. When the bug is straightforward or the LLM is confident based on prior knowledge, it may bypass certain tools. This flexible, integrated approach strengthens both the reasoning and solution phases, which eventually enhances the quality of generated patches.

In summary, this paper makes the following contributions:

- **Dimension.** This paper opens up a new dimension for LLM4APR research to incorporate repair ingredients to enhance the model's repair capabilities in the era of LLMs. Unlike recent LLM4APR work, which either uses only donor code [46] (i.e., internal repair ingredients) or only change actions [42] (i.e., external repair ingredients), our work effectively integrates both internal and external repair ingredients with LLMs to boost APR performance and alleviate the limitations of previous works. Our work showcases, for the first time, a promising path toward integrating LLM-based agents with repair ingredients.
- **Extensive Study.** We performed extensive evaluations on Defects4J V1.2 and Defects4J V2.0. The results demonstrate that ReinFix outperforms previous APR tools and further boosts the repair capabilities of LLMs. For instance, with GPT-3.5 as the foundational model, ReinFix successfully repairs 118 and 123 bugs in Defects4J V1.2 and V2.0, respectively, surpassing the top baseline APR tools by 4 and 16 bugs. Similarly, utilizing the more substantial GPT-4 model, ReinFix fixes 146 and 145 bugs in Defects4J V1.2 and V2.0, respectively, outperforming the best baselines by 32 and 38 bugs. Notably, ReinFix's application with GPT-4 fixed 61 (146-85) more bugs than the foundation model in our ablation study, marking a 71.76% improvement.
- **Open Science.** We have released ReinFix framework with two novel repair ingredients searching tools. Our repair ingredients retrieval scheme can be flexibly integrated into many LLM-based APR tools to further advance the APR community. We also encourage future researchers to leverage our approach to develop more powerful APR tools. Our source code and data are available at: https://sites.google.com/view/repairingredients.

2 Motivation

Despite the potential of LLMs in understanding code, they face challenges of lacking the ability to gather contextual clues and prior

src/com/google/javascript/jscomp/ControlFlowAnalysis.java

```
@@ -764,7 +764,7 @@ private static Node computeFollowNode(
     } else if (parent.getLastChild() == node) {
         if (cfa != null) {
             for (Node finallyNode : cfa.finallyMap.get(parent)) {
-                cfa.createEdge(fromNode, Branch.UNCOND, finallyNode);
+                cfa.createEdge(fromNode, Branch.ON_EX, finallyNode);
             }
         }
         return computeFollowNode(fromNode, parent, cfa);
```

(a) The human patch of Closure-14.

src/com/google/javascript/jscomp/ControlFlowGraph.java

```
@@ public static enum Branch {
    /** Unconditional branch. */
    UNCOND,
    /**
     * Exception-handling code paths.
     * Conflates two kind of control flow passing: …
     * In theory, we need 2 different edge types …
     */
    ON_EX,
```

(b) Internal repair ingredients for Closure-14.

Figure 1: A motivation example of Closure-14.

src/com/google/javascript/jscomp/CodeConsumer.java

```
@@ -238,7 +238,7 @@ void addNumber(double x) {
         add(" ");
     }

-    if ((long) x == x) {
+    if ((long) x == x && !isNegativeZero(x)) {
         long value = (long) x;
         long mantissa = value;
         int exp = 0;
```

(a) The human patch of Closure-51.

a retrieved bug-fix pair from the external repair history repository

```
@@ private int peekNumber() {
         }
     }
     if (last == NUMBER_CHAR_DIGIT && fitsInLong && (value != Long.MIN_VALUE
-        || negative)) {
     if (last == NUMBER_CHAR_DIGIT && fitsInLong && (value != Long.MIN_VALUE
+        || negative) && (value != 0 || !negative)) {
         peekedLong=negative ? value : -value;
         buffer.skip(i);
```

(b) External repair ingredients for Closure-51.

Figure 2: A motivation example of Closure-51.

knowledge of similar fixes (i,e., internal and external repair ingredients) independently to generate effective patches. In this section, we explore how these repair ingredients are critical to debugging and repair, using examples to show how missing ingredients can hinder effective patch generation.

2.1 Example 1: The Role of Internal Ingredients

Figure 1-(a) illustrates the Closure-14 bug from Defects4J, which occurs within Google's Closure Compiler project in the `ControlFlowAnalysis.java` file, specifically within the `computeFollowNode` method. This method is responsible for calculating the next node in control flow after a given node in an Abstract Syntax Tree (AST) is executed. However, the buggy code incorrectly uses `Branch.UNCOND`, an unconditional branch setting, to create edges for final nodes. This choice leads to an incorrect representation of the control flow because it fails to capture the exception-handling path correctly. The human patch corrects this by replacing `UNCOND` with `ON_EX`, ensuring the control flow graph accurately reflects that the "finally" block executes during exception propagation.

For a developer familiar with the Closure project, `Branch` and its attributes like `UNCOND` and `ON_EX` are meaningful, since they indicate specific control flow behaviors. However, without access to this internal information, the LLM faces challenges in understanding why `UNCOND` is incorrect in this context or why replacing it with `ON_EX` is effective. If the LLM only has access to the buggy function itself, synthesizing the correct patch becomes difficult due to the lack of contextual understanding about these parameters and their correct application. To solve this bug, essential donor code containing internal repair ingredients is needed. In this example, searching through the project reveals that the definition of `Branch` can be found in a non-buggy file, `ControlFlowGraph.java`. As illustrated in Figure 1-(b), by collecting and analyzing these internal repair ingredients, both within the buggy file and across other files in the project, the LLM can build a more comprehensive understanding of the needed context, ultimately leading to an effective fix.

2.2 Example 2: The Role of External Ingredients

Figure 2-(a) illustrates another bug, Closure-51. This bug reveals an incomplete handling of a floating-point edge case in which the code uses `(long) x == x` to verify whether a floating-point number x can be converted to a long integer without loss of precision. This condition works as intended for most values except x is equal to negative zero, allowing this specific value to proceed, and lead to unexpected behaviors and potential errors in subsequent code execution due to the peculiar properties of negative zero in floating-point arithmetic. The human-created patch addresses this by adding an additional condition, `!isNegativeZero(x)`, explicitly preventing negative zero from incorrectly passing the conversion condition. From a developer's perspective, the need for this patch is clear: without explicitly handling negative zero, the code risks unpredictable behaviors that can complicate program correctness.

However, this type of subtle bug presents a challenge for LLM-based approaches in APR. Despite the possibility that LLMs may have encountered similar edge cases in training, reliably generating the correct patch requires specialized domain knowledge about floating-point behavior and precision conversion. Smaller LLMs, in particular, might lack this depth of understanding. In this scenario, drawing on external repair history, which is knowledge about prior similar bugs and patches, could significantly enhance the LLM's ability to generate accurate fixes. For example, as illustrated in Figure 2-(b), repair history records containing analogous bug-fix pairs provide a valuable resource for understanding and addressing the current bug. Here, a repair history entry shows a bug fix with a similar structure, where a negative zero check is added to address a floating-point precision issue. These external repair ingredients offer concrete repair behaviors, guiding LLMs to craft effective patches for bugs by referencing prior solutions of similar issues.

3 Approach

In this section, we introduce ReinFix, which is designed to enhance LLM-based APR by incorporating both internal and external repair ingredients. As shown in Figure 3, ReinFix operates in two

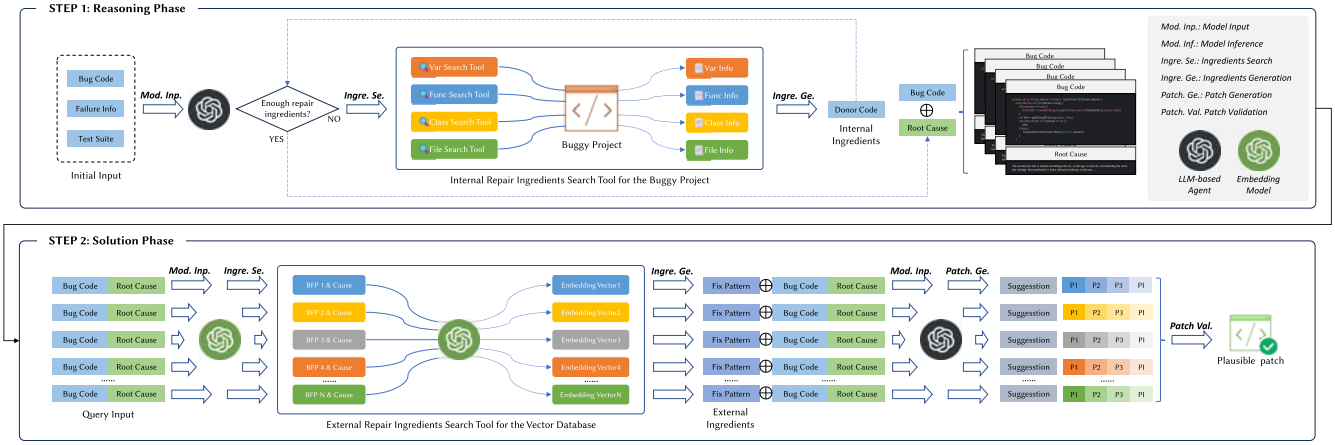


Figure 3: The workflow of ReinFix.

Table 1: The overview of code analysis tools.

Tool Name	Type	Description	Usage Example
identify_variable	Variable-Level	Analyzes variable definitions and usages in specific files	`varName, fileName`
find_variable_assignments	Variable-Level	Tracks all assignments to a specific variable	`varName, fileName`
track_variable_dataflow	Variable-Level	Analyzes data flow showing value sources and destinations	`varName, fileName`
trace_method_usage	Method-Level	Trace how and which file the method is used	`methodName`
analyze_method_details	Method-Level	Comprehensive method analysis	`methodName, fileName`
find_method_in_file	Method-Level	Analyzes the method control flow (if, while, for, etc.) structures	`methodName, fileName`
find_class_loc	Class-Level	Locate where the class is defined	`className`
identify_class	Class-Level	Retrieves class definition code including attributes and methods	`className`
get_imports	File-Level	Retrieves all import statements from a specific file	`fileName`

primary phases: reasoning and solution. In the reasoning phase, ReinFix supplies internal ingredients, which are derived through a toolkit that conducts dependency analysis to identify and retrieve relevant contextual code elements (e.g., definitions and usages of key variables or methods) within the project. This approach enables the LLM to gain a deeper understanding of project-specific dependencies and key identifiers, which aids in root cause analysis. In the solution phase, ReinFix enhances the patch generation by leveraging external ingredients. Specifically, the LLM retrieves similar examples from a vector database of bug-fix pairs based on the current bug and its root cause, using them as a reference for patch generation. The generated patches then undergo the validation process to output the plausible one which passes all test cases.

3.1 Internal Ingredient Search

In the reasoning phase, *internal ingredients* refer to critical code elements that aid in the bug cause comprehension. These include variables, methods, classes, and file dependencies essential for understanding the bug context and crafting an accurate patch. Existing works, such as *FitRepair* [46], rely on embedding relevant identifiers or variable names by performing similarity-based searches. While this approach can be effective, it has limitations when handling complex, cross-functional dependencies or cases requiring deeper contextual understanding. To address these challenges, ReinFix integrates a structured *dependency analysis* to identify and retrieve internal ingredients across various program structures.

Rather than embedding only variables or identifiers, we leverage *Joern* [16] to analyze the codebase through a *Code Property Graph (CPG)*. The CPG enables sophisticated queries across the code, helping gather multi-level dependencies for bug understanding. ReinFix uses Joern-based queries across four levels of analysis including *Variable*, *Method*, *Class*, and *File*, as shown in Table 1.

- **Variable-Level Analysis**: Variable-level tools focus on tracking variable definitions, usages, and types within specific files, providing precise information about how variables are defined and interact with other components. For example, the `identify_variable` tool enables targeted retrieval of variable definitions:

$$\text{identify\_variable}(\text{varName}, \text{fileName}) = \{v \mid v \in C_{\text{fileName}}, \text{is\_var}(v) \wedge \text{matches}(v, \text{varName})\}$$

  where C_{fileName} denotes all code elements in the file, returning matching variables by name. Additional tools like `find_variable_assignments` and `track_variable_dataflow` provide details on value assignments and data flow through variables, offering the LLM insights into how variables influence the program state.
- **Method-Level Analysis**: The `trace_method_usage` tool allows the retrieval of all method invocation locations, regardless of where they appear in the project:

$$\text{trace\_method\_usage}(\text{methodName}) = \{(f, l) \mid \text{invokes}(C_f, \text{methodName}) \text{ at location } l\}$$

  This query lists each location where `methodName` is invoked, providing control flow context essential for understanding program behavior. The `analyze_method_details` tool additionally extracts detailed control structures within the method, such as loops and exception handling, providing a richer basis for understanding how specific actions influence the program state.
- **Class-Level Analysis**: Classes represent higher-level structures that encapsulate variables and methods, and understanding class definitions can be vital in cases where multiple elements interact through shared methods or attributes. The `identify_class` tool can retrieve class definitions, including attributes and methods:

$$\text{identify\_class}(\text{className}) = \{c \mid c \in C_{\text{fileName}}, \text{is\_class}(c) \wedge \text{matches}(c, \text{className})\}$$

  This query retrieves the definition of `className`, giving the LLM insights into the encapsulated methods and properties that may influence the bug context. Similarly, the `find_class_loc` tool enables the LLM to pinpoint where the class is defined, helping to trace potential interactions and dependencies that are critical to understanding complex bug scenarios.

Searched internal repair ingredients for Closure-14.

```
cpg.identifier.name("Branch").typeFullName.take(1).l
val res1: List[String] = List("com.google.javascript.jscomp.ControlFlowGraph.Branch")

cpg.typeDecl.fullName("com.google.javascript.jscomp.ControlFlowGraph.Branch").astChildren.code.l
val res1: List[String] = List("""…
    /** Unconditional branch. */
  UNCOND,
  /**
   * Exception-handling code paths.
   * Conflates two kind of control flow passing:
   * - An exception is thrown, and falls into a catch or finally block
   * - During exception handling, a finally block finishes and control
   *   passes to the next finally block.
   * In theory, we need 2 different edge types. In practice, we
   * can just treat them as "the edges we can't really optimize".
   */
  ON_EX,
…""")
```

Figure 4: Searched class-level dependencies for Closure-14.

- **File-Level Analysis**: Since projects often span multiple files, file-level tools allow us to track dependencies, such as imports and inter-file references, that are crucial for understanding how components interact across files. For example, the `get_imports` tool gathers all imports in a file:

$$\text{get\_imports}(\text{fileName}) = \{i \mid i \in \text{imports}(C_{\text{fileName}})\}$$

  This information is essential when dependencies from external libraries or other modules impact the buggy code's behavior.

ReinFix extracts comprehensive internal ingredients from the codebase, enabling the LLM to go beyond simple identifier matching and to trace complex dependencies that often hold the key to uncovering a bug's root cause. By thoroughly analyzing variable definitions, method interactions, and control flows, ReinFix equips the LLM with a holistic understanding of how different components interact, which is crucial for identifying where and why errors arise. For instance, in *Closure-14* (Figure 1), the bug stems from an incomplete representation of control flow in the `computeFollowNode` method, where the code mistakenly uses `Branch.UNCOND` instead of `ON_EX`. As shown in Figure 4, by leveraging the `find_class_loc` tool and `identify_class tool`, the LLM can retrieve information on where class `Branch` locates and search for the definition code of this class, including comments of the attributes. It is clear that `ON_EX` from other parts of the project, although this identifier is absent from the buggy code. This allows the LLM to identify that `ON_EX` is the correct control flow branch to use in exception propagation, illuminating the underlying root cause.

3.2 External Ingredient Search

In the solution phase, ReinFix incorporates external repair ingredients derived from a large corpus of historical bug-fix pairs. These external ingredients, consisting of buggy code and their corresponding fixes, are essential to provide the LLM with relevant repair knowledge. Using this historical corpus, ReinFix can offer insights into common bug patterns, allowing the LLM to reference and adapt known solutions to current bugs. This is crucial as many software bugs exhibit recurring patterns, and leveraging previous repairs can greatly improve the patch efficiency and accuracy [11, 28, 31, 32, 42]. However, directly adopting the standard Retrieval-Augmented Generation (RAG), such as RAP-Gen [42], which focuses on identifying similar buggy code, then retrieving the corresponding fix. While this is effective in some cases, it neglects the deeper context-specifically, the root cause of the bug. Bugs of similar symptoms might have different underlying causes, and relying solely on code similarity does not guarantee the identification of an appropriate fix.

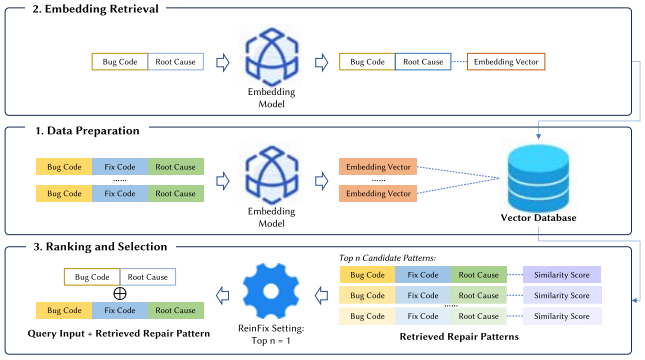


Figure 5: The workflow of retrieving external repair patterns through the enhanced RAG paradigm.

To overcome these limitations, we enhance the corpus of BFPs by generating concise causes via prompting an LLM (e.g., GPT-4). For each buggy code and patch pair, forming structured triads of (buggy_code, fix_code, root_cause). This augments traditional RAG by integrating both buggy code and root cause into the retrieval process, ensuring retrieved repair patterns are contextually and semantically relevant. The retrieval process, shown in Figure 5, comprises three stages: Data Preparation, Embedding Retrieval, and Ranking and Selection.

- **Data Preparation.** First, we collect a large amount of historical bug-fix pairs from open-source code repositories and use LLMs (e.g., we use GPT-4o in our experiment) to automatically label the root cause for each BFP. Then we load each (*buggy_code*, *fix_code*, *root_cause*) triad from the corpus. For efficient retrieval, we preprocess the corpus by embedding each buggy code and its root cause. Let each buggy code c_i and its corresponding root cause r_i be part of a pair (c_i, r_i) from the corpus. We concatenate c_i and r_i into a single input string and transform it into a fixed-length embedding vector e_i using a pre-trained text embedding model (e.g., *text-embedding-3-large* [40]):

$$e_i = f_{\text{embed}}(c_i \oplus r_i) \tag{1}$$

  where $\oplus$ denotes the concatenation operation. These embeddings e_i are stored in a vector database alongside the original corpus, resulting in a structure of (*buggy_code*, *fix_code*, *root_cause*, *embedding_value*). This embedded structure enables fast and accurate similarity comparisons for retrieving relevant repair patterns during the retrieval phase.
- **Embedding Retrieval.** Upon receiving a query in the form of "<buggy code, root cause>", the query is also transformed into an embedding vector q, which combines the buggy code and the root cause from the reasoning phase:

$$q = f_{\text{embed}}(\textit{query\_code} \oplus \textit{root\_cause}) \tag{2}$$

  This query embedding is then compared to the embeddings in the precomputed corpus using cosine similarity:

$$\text{similarity}(q, e_{embedding}) = \frac{q \cdot e_{embedding}}{\|q\| \|e_{embedding}\|} \tag{3}$$

  The goal is to find historical bug-fix pairs that share similar structural and semantic characteristics with the query.

Top-1_External_Ingredients_Closure-51.json

```
{"Buggy Code":
"
private int peekNumber() {
…
// bug_start
     if (last == NUMBER_CHAR_DIGIT && fitsInLong && (value != Long.MIN_VALUE || negative)) {
// bug_end
…
",
"Fixed Code":
"
// fix_start
     if (last == NUMBER_CHAR_DIGIT && fitsInLong && (value != Long.MIN_VALUE || negative) &&
(value != 0 || !negative)) {
// fix_end
",
"Root Cause":
"The condition fails to ensure that the value is not zero when negative is true, which leads to
an incorrect handling of the number -0.",
"similarity": 0.625547582462616}
```

Figure 6: Retrieved repair pattern for *Closure-51*.

- **Ranking and Selection.** The top n results are retrieved based on similarity scores. For each retrieved entry, the corresponding buggy code, fix code, and root cause are returned to the LLM along with the similarity score. These retrieved patterns serve as potential repair behaviors that the LLM can use to generate a correct patch for the current bug. Formally, let R denote the set of top n entries:

$$R = \{e \in \text{corpus} \mid \text{similarity}(q, e_{embedding}) \geq \text{threshold}\} \quad (4)$$

  The LLM then uses these repair behaviors to inform its patch generation, ensuring that the proposed fix is grounded in prior successful repairs.

In the *Closure-51* (Figure 2), ReinFix first queries the bug corpus using the buggy code and a preliminary root cause. The retrieved repair pattern, shown in Figure 6, matches the root cause related to negative zero and suggests an appropriate fix. By referring to this repair pattern, ReinFix is also able to generate a patch that addresses the underlying issue. This process demonstrates how combining both buggy code and root cause information improves the relevance of the retrieved repair patterns.

3.3 Overall Workflow

ReinFix was built on the ReAct (Reasoning and Action) framework, completing adaptive and context-aware program repair through *Thought-Action-Observation* cycles. In the **Thought Cycle**, the agent identifies which ingredients are essential for the bug comprehension. In the **Action Cycle**, where the agent invokes relevant tools based on the Thought Cycle's reasoning, conducting contextual code element analysis and external ingredient retrieval. Then, the agent interprets these tools' outputs to integrate new knowledge into its repair context during **Observation Cycle**, deciding on whether to use this information to refine its search or, if sufficient, proceed toward patch generation. Through this iterative process, the agent reasons about necessary steps, selectively calls tools, and interprets tool outputs to make informed decisions. This structured process adaptively searches internal codebase and external bug-fix patterns, efficiently synthesizing insights for patch generation.

Algorithm 1 outlines the overall workflow of ReinFix. *1) Reasoning Phase*: This phase involves the agent's primary exploration of root causes and gathering relevant repair ingredients (lines 3-5). If the initial root cause is inconclusive, the agent initiates an iterative search by invoking tools to collect contextual elements for reasoning (lines 6-22), constantly refining its analysis until the information is sufficient for analyzing the root cause. *2) Solution Phase*: The agent enters to this phase once the necessary repair ingredients are obtained. Here, it synthesizes a repair plan by integrating the internal and external insights (lines 23-25). The agent assembles the root cause, buggy code, and relevant repair patterns to generate repair suggestions. These suggestions form the basis of candidate patches, which are then validated against test cases to ensure reliability and relevance (lines 26-27).

Figure 7 illustrates the sequential Thought-Action-Observation cycle of the ReAct Agent, showing how it derives the root cause and repair suggestions for a bug. The agent reads the bug information and follows the Algorithm 1's analysis process to address complex tasks while avoiding potential infinite loops during reasoning and action. After gathering all relevant contextual code elements and learning from matched repair patterns, the agent provides a detailed explanation of the root cause, along with several repair suggestions as the Final Answer, which guides the subsequent patch generation process. For each root cause and suggested fix pair, we ask the model to generate multiple patches. These patches are then evaluated by running them against the test suite. Following previous works [2, 49, 54], patches that pass all test cases are considered *plausible*

Algorithm 1: ReinFix

```
1  Framework ReinFix:
       Input  : bugCode (buggy function code), failureInfo (failing test info),
                testSuite (test suite)
       Output : pPatch (plausible patch)
2      Start Reasoning Phase:
3          rootCause ← CauseAnalyzewithoutDonor(bugCode, failureInfo,
             testSuite)
4          if rootCause is not NONE then
5              return rootCause
6          else
7              Open CPG Project
8              donorCode, varInfo, funcInfo, classInfo, fileInfo
                 ← ∅, ∅, ∅, ∅, ∅
9              isEnoughDonor ← False
10             analyze_variable_tool, analyze_method_tool,
                 analyze_class_tool, analyze_file_tool ← toolkit
11             while isEnoughDonor is False do
12                 varName, funcName, className, fileName ←
                     ingredientsSelection(bugCode, donorCode)
13                 varInfo ← analyze_variable_tool(varName)
14                 funcInfo ← analyze_method_tool(funcName)
15                 classInfo ← analyze_class_tool(className)
16                 fileInfo ← analyze_file_tool(fileName)
17                 donorCode ← donorCode ∪ {varInfo, funcInfo, classInfo,
                     fileInfo}
18                 rootCause ← causeanalyzewithDonor(bugCode,
                     donorCode, failureInfo, testSuite)
19                 if rootCause is not NONE then
20                     Close CPG Project
21                     isEnoughDonor ← True
22             return rootCause
23     Start Solution Phase:
24         queryInput ← concatenateInfo(bugCode, rootCause)
25         repairPattern ← repair_pattern_retrieval_tool(queryInput)
26         repairsSuggestions ← giveRepairSuggestions(bugCode, rootCause,
             repairPattern)
27         candidatePatches ← patchGeneration(bugCode, repairSuggestions)
28         pPatch ← patchValidation(candidatePatches)
29         return pPatch
```

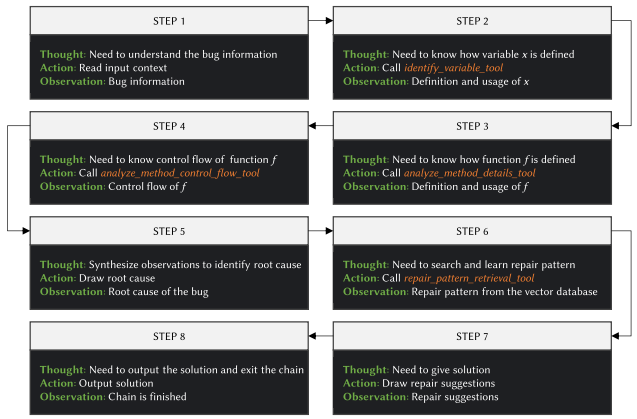


Figure 7: An execution chain of the ReAct Agent.

patches, while those verified by humans and deemed semantically equivalent to the ground truth are classified as *correct patches*.

4 Experiment Setup

4.1 Research Questions

- **RQ1: How does the repair effectiveness of ReinFix compare to state-of-the-art APR techniques?** This question evaluates ReinFix's repair capabilities in Java bug repair and compares its performance to existing baselines. (Repair Effectiveness)
- **RQ2: How does ReinFix perform across different repair scenarios?** This question investigates ReinFix's performance in three distinct repair scenarios [49, 54], e.g., single-function, single-hunk, and single-line bugs. (Repair Scenarios)
- **RQ3: How do different design choices in ReinFix impact its repair capabilities?** This question explores how ReinFix's novel repair ingredients search strategies affect the overall repair performance through an ablation study. (Ablation Study)
- **RQ4: Can ReinFix generalize to recent bugs post-LLM training data cut-off?** This question assesses ReinFix's generalization ability by testing its performance on recent real-world bugs, ensuring no data leakage. (Generalizability Study)

Table 2: Statistics of Defects4J.

Benhmarks	Defects4J V1.2				Defects4J V2.0			
Repair Scenarios	# MF Bugs	# SF Bugs	# SH Bugs	# SL Bugs	# MF Bugs	# SF Bugs	# SH Bugs	# SL Bugs
# Bug Num	136	255	154	80	210	228	159	78

4.2 Benchmarks

- **Defects4J.** To evaluate the repair effectiveness, we use the widely studied benchmark Defects4J [17]. Similar to prior APR studies [49, 54], we separate Defects4J into V1.2 and V2.0. Defects4J V1.2 consists of 391 bugs (removed 4 deprecated bugs), and Defects4J V2.0 introduces 438 new bugs. In this work, we follow prior studies [49, 54] and categorize Defects4J into three repair scenarios, including single-function (SF), single-hunk (SH), and single-line (SL) bugs. Note that single-hunk is a subset of single-function and single-line is a subset of single-hunk bugs. In addition, we will also implement fixes on the multi-function (MF) scenario. The statistic of each scenario is presented in Table 2.
- **RWB.** To evaluate the generalizability study, we use the recent repair benchmark RWB [54]. Note that the pre-training data for ChatGPT (GPT-3.5 [35]) was collected before September 2021 [37], and the pre-training data for DeepSeek-Coder was collected from GitHub before February 2023 [1]. To assess data leakage of ChatGPT and DeepSeek, Yin *et al.* [54] collected two datasets to assess data leakage. The first dataset (RWB V1.0) comprises bug-fixing commits after October 2021, while the second dataset (RWB V2.0) includes bug-fixing commits after March 2023, resulting 44 and 29 single-function bugs, respectively.

4.3 Baselines

We considered recent APR works to serve as baselines. This includes: ChatRepair [49], ThinkRepair [54], RepairAgent [2], FitRepair [46], GAMMA [60], TENURE [32], Tare [62], AlphaRepair [48], RAPGen [42], KNOD [14], Recoder-T [61, 62], TBar [28]. Following the common practice in the APR community [42, 46, 48, 49, 54, 60–62], we reuse the reported results from previous studies [14, 32, 42, 46, 48, 49, 54, 60, 62] instead of directly running the APR tools.

4.4 Vector Database

To construct a vector database for the external ingredients search, we utilize the TRANSFER dataset [31] as the retrieval corpus and the text-embedding-3-large [40] as the embedding model. TRANSFER dataset contains about one million BFPs and corresponding fix templates that have been used to drive the template-based APR works [11, 31, 32]. Considering the cost of embedding, we randomly selected 100K samples from TRANSFER for the vector database construction. We use the exact match strategy to filter out overlapping samples with those in benchmarks to avoid data leakage.

4.5 Implementation

ReinFix is built on top of the LangChain [20] framework, where we use gpt-3.5-turbo [35] and gpt-4o [39] as base models, and set a sampling temperature of 1. In fault localization (FL), to avoid additional biases introduced by the FL tool, we follow recent works [49, 54] under conditions of perfect fault localization. When generating patches, we limit the number of repair attempts to a maximum of 3 per bug. For each attempt, the maximum number of repair suggestions is set to 3, and for each repair suggestion, up to 5 candidate patches are considered. Therefore, the total maximum patch space size is 3*3*5=45. For validation, only the top-1 plausible patch that passes all test cases is retained. Finally, the plausible patches are manually reviewed to verify the semantic correctness.

5 Evaluation

5.1 RQ1: Repair Effectiveness

In the main experiment, we will investigate the overall repair effectiveness of ReinFix on the Defects4J. Table 3 presents the repair results of the specific implementation of ReinFix.

Defects4J V1.2. As shown in Table 3, the implementations of ReinFix outperform the current best baseline work, the ChatGPT-based APR tool ChatRepair [49]. Specifically, the GPT3.5-based ReinFix$_{GPT3.5}$ fixes 4 more bugs (118 vs. 114) than ChatRepair, and the GPT4o-based ReinFix$_{GPT4o}$ fixed 32 more bugs (146 vs. 114)

Table 3: Repair results (correct fixes / plausible fixes) for ReinFix and baselines on Defects4J V1.2 and V2.0.

APR Tool	ReinFix$_{GPT4o}$	ReinFix$_{GPT3.5}$	ChatRepair	ThinkRepair	RepairAgent	FitRepair	GAMMA	TENURE	Tare	AlphaRepair	RAP-Gen	KNOD	Recoder	TBar
Sampling Times	3*3*5	3*3*5	500	25*5	117	1000*4	250	500	100	5000	100	1000	100	-
Chart	18/20	16/17	15/-	11/-	11/14	8/-	11/11	7/-	11/-	9/-	9/-	10/11	9/-	11/-
Closure	40/50	30/37	37/-	31/-	25/25	29/-	24/26	26/-	25/-	23/-	22/-	23/29	25/-	16/-
Lang	33/47	26/33	21/-	19/-	17/17	19/-	16/25	16/-	14/-	13/-	12/-	11/13	12/-	13/-
Math	39/68	35/52	32/-	27/-	29/29	24/-	25/31	22/-	22/-	21/-	26/-	20/25	20/-	22/-
Mockito	10/11	8/9	6/-	6/-	6/6	6/-	3/3	4/-	2/-	5/-	2/-	5/5	2/-	3/-
Time	6/11	3/4	3/-	4/-	2/3	3/-	3/5	4/-	3/-	3/-	1/-	2/2	3/-	3/-
#Total (D4J V1.2)	**146/207**	**118/152**	114/-	98/-	90/94	89/-	82/101	79/-	77/-	74/109	72/-	71/85	71/-	68/95
#Total (D4J V2.0)	**145/190**	**123/147**	48/-	107/-	74/92	44/-	45/-	50/-	-/-	36/-	53/-	50/85	11/-	8/-

Table 4: Repair results (correct fixes) of different repair scenarios for ReinFix and baselines on Defects4J V1.2 and V2.0.

Benhmarks	Defects4J V1.2				Defects4J V2.0			
Repair Scenarios	# MF Bugs	# SF Bugs	# SH Bugs	# SL Bugs	# MF Bugs	# SF Bugs	# SH Bugs	# SL Bugs
ChatRepair	-	76	-	-	-	-	-	48
ThinkRepair	-	98	78	52	-	107	81	47
RepairAgent	7	83	71	51	6	68	65	48
ReinFix$_{GPT3.5}$	14	104	78	53	14	109	85	47
ReinFix$_{GPT4o}$	22	124	93	57	15	130	103	56

than ChatRepair. In particular, under the same condition of using GPT-3.5 as the foundation model, ReinFix still outperforms the recent approaches ChatRepair and ThinkRepair, fixing 4 and 20 more bugs, respectively. This indicates that ReinFix's strategic design not only provides a competitive edge over similar methods but also more effectively harnesses the repair potential of the LLM. It is important to note that ThinkRepair is specifically designed for the single-function repair scenario, where its reported results include 98 successfully fixed single-function bugs. In contrast, the repair outcomes for ReinFix and ChatRepair encompass both single-function and multi-function bugs. To provide a more direct comparison with the state-of-the-art single-function repair tool, ThinkRepair, we focus on the single-function repair scenario. In this context, ReinFix$_{GPT3.5}$ successfully repairs 104 single-function bugs, surpassing ThinkRepair by 6 cases. Overall, ReinFix not only outperforms the best-performing baseline, ChatRepair, in terms of overall repair effectiveness, but also performs on par with ThinkRepair in the single-function repair scenario.

Defects4J V2.0. As shown in Table 3, ReinFix again demonstrates superior performance compared to the current leading baseline, the ChatGPT-based APR tool ThinkRepair [49]. Specifically, the GPT3.5-based ReinFix$_{GPT3.5}$ fixes 16 more bugs (123 vs. 107) than ThinkRepair, and the GPT4o-based ReinFix$_{GPT4o}$ fixed 38 more bugs (145 vs. 107) than ThinkRepair. As mentioned previously, unlike ReinFix, which aims to address all bug types, ThinkRepair [54] is specifically designed for single-function repairs, while ChatRepair [49] and other baselines [46, 48] focus only on single-line repair scenarios. To ensure a fair comparison with these recent GPT-3.5-based APR tools, we further analyze the repair capabilities of ReinFix$_{GPT3.5}$ within single-function and single-line repair scenarios. In the single-function repair scenario, ReinFix$_{GPT3.5}$ successfully fixes 2 more bugs than ThinkRepair (109 vs. 107). In the single-line repair scenario, ReinFix$_{GPT3.5}$ also rivals ChatRepair's repair results (47 vs. 48). Overall, ReinFix not only demonstrates an outstanding repair capability but also rivals or exceeds recent LLM-based APR methods in both single-function and single-line repair scenarios.

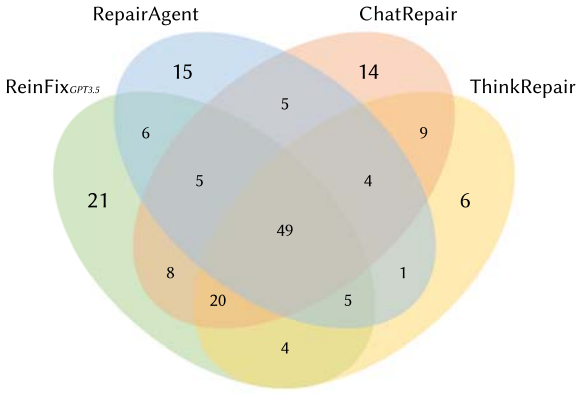


Figure 8: Bug fix Venn diagram on Defects4J V1.2.

Unique Fixes. Specifically, we also present ReinFix's unique repair capabilities compared to other APR tools. Specifically, we follow the practice of selecting Defects4J V1.2 in the baseline work [49] to present unique fixes and keep the same base model GPT3.5 as recent LLM-based baselines [2, 49, 54]. As shown in Figure 8, ReinFix$_{GPT3.5}$ still maintains an advantage in unique repair capabilities, achieving 21 unique fixes compared to recent APR tools using the same base model. These results indicate that ReinFix's approach is complementary to existing work, emphasizing the necessity of searching for repair ingredients to enhance existing APR workflow.

In summary, ReinFix achieves the best bug repair performance on both Defects4J V1.2 and V2.0. In particular, ReinFix maintains an advantage over recent approaches when using the same foundation model. These results highlight the effectiveness of ReinFix's repair strategy and suggest that it represents a promising direction.

5.2 RQ2: Repair Scenarios

While the main experiments provide an overview of ReinFix's overall repair effectiveness, examining its behavior within specific scenarios such as single-line, single-hunk, and single-function, allows us to assess its performance across different dimensions of complexity. Therefore, here we explore the multidimensional capabilities of ReinFix across various repair scenarios [54]. By analyzing each repair scenario individually, following recent work [49, 54], we aim to reveal ReinFix's capabilities in greater detail and across multiple facets of the repair process.

As shown in Table 4, ReinFix achieves effective repair results across all tested scenarios, even without considering multi-function repair cases. Specifically, in the single-function repair scenario,

ReinFix$_{GPT3.5}$ and ReinFix$_{GPT4o}$ repaired 6 and 26 more single-function bugs than the top-performing baseline, ThinkRepair [54], on Defects4J V1.2, and 2 and 23 more single-function bugs, respectively, on Defects4J V2.0. In the single-hunk and single-line repair scenarios, ReinFix$_{GPT3.5}$ and ReinFix$_{GPT4o}$ continue to match or even exceed recent baseline performances. More importantly, we assessed ReinFix in the multi-function repair scenario, comparing it with RepairAgent [2], another tool supporting multi-function fixes. Here, ReinFix$_{GPT3.5}$ and ReinFix$_{GPT4o}$ again demonstrated higher fix rates. Overall, these findings indicate that ReinFix exhibits robust performance across diverse repair scenarios.

5.3 RQ3: Ablation Study

In the ablation experiment, we aim to evaluate the contributions of ReinFix's key components in enhancing the repair capabilities of LLMs and to analyze the advantages of our repair ingredient retrieval methods (i.e., Section 3.1 and Section 3.2) in comparison to previous work. Therefore, we designed several variants to reveal the effects of different design choices. Specifically, we created six ReinFix variants: ❶ ReinFix$_{NN}$: We removed two key components from the original ReinFix implementation to establish a basic baseline that represents the fundamental repair capabilities of ChatGPT. ❷ ReinFix$_{DN}$: In this variant, we investigate the impact of ReinFix's Dependency-Analysis-Based internal ingredient search approach (Section 3.1) on repair effectiveness. Only the internal ingredient search component is used for contextual code search, assisting LLMs in better analyzing the root cause of the bugs. ❸ ReinFix$_{NP}$: This variant explores the impact of ReinFix's Pattern-Matching-Based external ingredients search approach (Section 3.2) on repair effectiveness. Only the external ingredient search component is used for bug pattern retrieval, assisting LLMs in better identifying repair behaviors to generate patches. ❹ ReinFix$_{FP}$: This variant explores the advantages of ReinFix's dependency-analysis-based internal ingredient search approach over the search schemes used in recent work [46]. The original internal ingredient search component of ReinFix is replaced with FitRepair's repair ingredient retrieval scheme to observe the impact of using different contextual code search strategies on the repair results. ❺ ReinFix$_{DR}$: To explore the advantages of ReinFix's pattern-matching-based external ingredient search approach over the retrieval schemes used in recent work [42], we replaced ReinFix's original external ingredients search component with RAP-Gen's repair ingredient retrieval scheme. This allows us to observe the impact of using different search strategies on the repair results. ❻ ReinFix$_{DP}$: For reference, we include the original implementation of ReinFix with both key repair ingredients search components, showcasing the full capabilities of the system.

In this ablation study, we select GPT-4o as the foundational model and Defects4J V1.2 as the benchmark for testing. We will now analyze the contributions and advantages of the design choices in ReinFix by evaluating several variants of the system.

5.3.1 Contributions of ReinFix in Design Choices. By comparing the repair results of the ReinFix variants (❶, ❷, ❸, and ❻), we can examine the contributions of ReinFix's key components and evaluate their roles in enhancing the repair capabilities of LLMs.

Table 5: Repair results (correct fixes) of ReinFix when using different ingredients search components on Defects4J V1.2.

Variants	Repair Ingredients Search Components		Repair Result
	Internal Ingredients Search	External Ingredients Search	
❶ ReinFix$_{NN}$	NULL	NULL	85
❷ ReinFix$_{DN}$	Dependency-Analysis-Based	NULL	116
❸ ReinFix$_{NP}$	NULL	Pattern-Matching-Based	108
❹ ReinFix$_{FP}$	FitRepair [46]	Pattern-Matching-Based	110
❺ ReinFix$_{DR}$	Dependency-Analysis-Based	RAP-Gen [42]	119
❻ ReinFix$_{DP}$	Dependency-Analysis-Based	Pattern-Matching-Based	146

1) ❶ ReinFix$_{NN}$ vs. ❷ ReinFix$_{DN}$: The first key component of the ReinFix is to help LLM reason about possible bug root causes by internal ingredient search. By comparing the repair results of ReinFix$_{NN}$ and ReinFix$_{DN}$, we can directly observe the impact of including the first key component on the repair capability of LLMs. As shown in Table 5, ReinFix$_{DN}$ fixes 31 more bugs (116 compared to 85) than ReinFix$_{NN}$. This result indicates that the first key component is crucial for enhancing the repair capability of LLMs. By reasoning about the root cause of bugs and retrieving relevant internal donor code through the dependency-analysis based approach, the LLM gains a more comprehensive understanding of the bugs, which enables it to generate more accurate fix patches and achieve better repair results.

2) ❶ ReinFix$_{NN}$ vs. ❸ ReinFix$_{NP}$: The second key component of the ReinFix is to help LLM generate correct repair actions by external ingredient search. By comparing the repair results of ReinFix$_{NN}$ and ReinFix$_{NP}$, we can directly observe the impact of including the second key component on enhancing the repair capability of LLMs. Since ReinFix$_{NP}$ removes the first key component, the LLM instead independently reasons about the root cause of the bug to complete the fix. As shown in Table 5, ReinFix$_{NP}$ fixes 23 more bugs (108-85) than ReinFix$_{NN}$, indicating that the second key component is also crucial for enhancing the repair capability of LLMs. By generating correct repair patches through the search for relevant repair behaviors, the LLM can enrich its fix-related domain knowledge, leading to improved repair results.

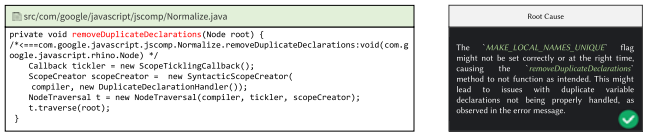
3) ❻ ReinFix$_{DP}$ vs. ❶ ReinFix$_{NN}$ ❷ ReinFix$_{DN}$ ❸ ReinFix$_{NP}$: It is important to note that the complete implementation of ReinFix is a two-phase workflow. In the root cause analysis (i.e., reasoning) phase, it searches for internal repair ingredients to help better understand the root cause of the bug. In the repair pattern matching (i.e., solution) phase, it uses the root cause obtained in the previous phase to help search for relevant repair actions and implement patch generation. Indeed, there is a dependency between the two key components of ReinFix and their synergies also deserve attention. As shown in Table 5, ReinFix$_{DP}$ not only fixes 61 more bugs than ReinFix$_{NN}$ but also fixes 30 and 38 more bugs than ReinFix$_{DN}$ and ReinFix$_{NP}$, respectively. These results demonstrate that both key components of ReinFix are crucial for enhancing the base repair capability of LLMs, and using them together maximizes the overall repair effectiveness.

5.3.2 Advantages of ReinFix in Design Choices. By comparing the results of ReinFix variants ❹ ❺ ❻, we can reveal the advantages of our repair ingredients search methods over previous works [42, 46].

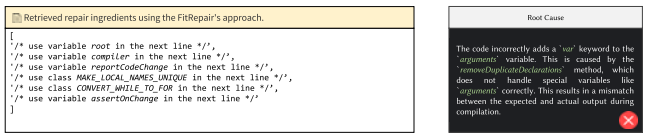
src/com/google/javascript/jscomp/Normalize.java

```
@@ -86,12 +86,12 @@ private void reportCodeChange(String changeDescription) {
     @Override
     public void process(Node externs, Node root) {
         NodeTraversal.traverse(compiler, root, this);
+        removeDuplicateDeclarations(root);
         if(MAKE_LOCAL_NAMES_UNIQUE) {
             MakeDeclaredNamesUnique renamer = new MakeDeclaredNamesUnique();
             NodeTraversal t = new NodeTraversal(compiler, renamer);
             t.traverseRoots(externs, root);
         }
-        removeDuplicateDeclarations(root);
         new PropogateConstantAnnotations(compiler,assertOnChange).process(externs, root);
     }
```

Figure 9: The human patch of Closure-102.



(a) Retrieved repair ingredients of REINFIX$_{DP}$.



(b) Retrieved repair ingredients of REINFIX$_{FP}$.

Figure 10: The Repair process of REINFIX$_{DP}$ and REINFIX$_{FP}$ for Closure-102.

Table 6: Repair results (correct fixes) for REINFIX and baselines on RWB V1.0 (44 cases) and V2.0 (29 cases).

Benchmark	RWB V1.0 (44 bugs)				RWB V2.0 (29 bugs)			
APR Tool	REINFIX$_{GPT4}$	REINFIX$_{GPT3.5}$	ThinkRepair	AlphaRepair	REINFIX$_{GPT4t}$	REINFIX$_{DSC}$	ThinkRepair*	AlphaRepair
LLM	GPT-4	GPT-3.5	GPT-3.5	CodeBERT	GPT-4-turbo	DeepSeek	DeepSeek	CodeBERT
Cli	4	4	4	3	4	4	4	3
Codec	3	3	3	1	3	2	1	1
Collections	1	1	1	0	-	-	-	-
Compress	2	2	1	1	1	1	-	-
Csv	1	1	1	0	-	-	-	-
Jsoup	7	6	6	2	4	2	2	1
Lang	3	3	3	2	3	3	3	1
# Total	**21**	**20**	19	9	**15**	**12**	10	6

1) ❻ REINFIX$_{DP}$ vs. ❹ REINFIX$_{FP}$: In the first key component design of REINFIX, we adopted a program-dependency-based approach to search for internal donor code repair ingredients to assist in root cause analysis. A closely related work, FitRepair [46], employs a code-similarity-based approach [21] to retrieve relevant identifiers for prompt augmentation. While REINFIX and FitRepair differ in their respective purposes for leveraging repair ingredients, they also adopt distinct design choices for searching these ingredients. To fairly evaluate the advantages of REINFIX's design choices over those of FitRepair, we re-implemented the first key component of REINFIX using FitRepair's similarity-based approach for direct comparison. As shown in Table 5, REINFIX$_{DP}$ fixed 36 more bugs than REINFIX$_{FP}$, demonstrating the superiority of REINFIX's donor code search strategy over FitRepair's approach. Upon analysis, we found that repair ingredient retrieval methods that do not consider program dependencies often introduce irrelevant or incorrect donor code. This can mislead the model into inferring incorrect root causes and result in repair failures. By leveraging program dependencies, REINFIX ensures the retrieval of contextually accurate ingredients, which is critical for effective bug fixing. Here, we observe the case, Closure-102 (Figure 9), where REINFIX$_{DP}$ successfully repaired the bug, while REINFIX$_{FP}$ failed. As shown in Figure 10-(a), REINFIX$_{DP}$ accurately identifies the definition of the buggy method during the reasoning phase by leveraging program dependency analysis, enabling the model to infer the correct root cause. Conversely, as illustrated in Figure 10-(b), REINFIX$_{FP}$ retrieves repair ingredients based solely on code similarity, resulting in irrelevant donor code. These irrelevant ingredients fail to assist the model in analyzing the critical root cause, causing it to produce incorrect fixes.

2) ❻ REINFIX$_{DP}$ vs. ❺ REINFIX$_{DR}$: In the second key component design of REINFIX, we adopted a pattern matching-based approach to search external repair ingredients for similar repair patterns. The closest work is RAP-Gen [42], which also follows the RAG paradigm to retrieve relevant fix patterns for prompt augmentation. The key difference is that RAP-Gen's method does not consider high-level bug root cause information. To fairly evaluate the advantages of REINFIX's design choices over RAP-Gen, we reimplemented the second key component of REINFIX using RAP-Gen's retrieval approach for direct comparison. As shown in Table 5, REINFIX$_{DP}$ fixed 27 more bugs than REINFIX$_{DR}$, indicating that REINFIX's search strategy outperforms RAP-Gen's approach. We analyzed the reasons behind it and found that relying solely on source code-level (the function) similarity is insufficient, as similar code may have different bugs. In contrast, by incorporating bug root cause information, REINFIX effectively guides the retrieval of relevant repair actions. This highlights the importance of considering the root cause in repair pattern retrieval, as bugs with similar root causes are likely to require similar repair behaviors. For example, in Closure-51 (Figure 2), REINFIX$_{DP}$ retrieved a relevant repair behavior `(value!=0 || !negative)`, whereas REINFIX$_{DR}$ retrieved an irrelevant repair behavior `if (pos-start >= 10)`. This highlights that high-level root cause is critical not only for understanding the bug but also for retrieving accurate repair behaviors.

5.4 RQ4: Generalizability Study

In the generalizability experiments, we focus on the performance of the REINFIX on additional benchmarks as well as other foundation models. Here, we follow ThinkRepair [54] to test on the RWB benchmark and implement REINFIX$_{DSC}$ with DeepSeek-Coder [1] as the foundation model. Note that the RWB V1.0 and RWB V2.0 collect bug cases after the training cutoff date for GPT-3.5 and DeepSeek-Coder, respectively, and thus the RWB dataset can further mitigate the data leakage risks of LLMs during the repair process. As shown in Table 6, REINFIX$_{GPT3.5}$ fixes 1 more bug than ThinkRepair on RWB V1.0, and REINFIX$_{DSC}$ fixes 2 more bugs than ThinkRepair* on RWB V2.0. Overall, REINFIX achieves a 10.34% improvement over ThinkRepair. These results demonstrate that even when minimizing the risk of data leakage and using the same base model, REINFIX remains effective and outperforms the best baseline.

Additionally, we tested several REINFIX implementations using other LLMs to explore whether our framework remains effective across different models. Specifically, considering the training data cutoff for RWB V1.0 and V2.0, we selected GPT-4 (i.e., gpt-4-0613 [36]) and GPT-4-turbo (i.e., gpt-4-1106-preview [38]) to evaluate the framework on these two benchmark versions, respectively.

As shown in Table 6, ReinFix$_{GPT4}$ and ReinFix$_{GPT4t}$ fixed 21 and 15 bugs on RWB V1.0 and V2.0, respectively, achieving the best repair results. These results highlight that the ReinFix framework demonstrates strong generalization, effectively producing fixes not only on additional benchmarks but also across a range of LLMs.

6 Threats to Validity

Data Leakage. The training data of LLMs may contain correct patches for buggy projects, potentially affecting the evaluation. To mitigate this risk, we follow previous work [54] and evaluate on bug cases collected after the cutoff date for LLM training data to ensure that the LLMs have not encountered the bug cases in the testing benchmarks during training. As shown in the generalizability experiments (Section 5.4), ReinFix performs well on the recently collected RWB benchmarks, which helps minimize the impact of data leakage on the results. Moreover, our ablation experiments (Section 5.3) demonstrate that the design choices in ReinFix further enhance the repair capability of base models, ensuring that the observed gains are independent of data leakage.

Repair Costs. Using LLMs may lead to higher repair costs. To address this concern, we follow prior work [49] that evaluates the average cost per bug fix. In our main experiment, ReinFix$_{GPT3.5}$, which uses GPT-3.5 as the base model, incurs an average cost of $0.06 per bug fix, while ReinFix$_{GPT4o}$ with GPT-4o as the base model averages $1.45 per bug fix. In comparison, ChatRepair, using GPT-3.5 as the base model, has an average cost of $0.42 per bug fix. Thus, ReinFix$_{GPT3.5}$ not only reduces costs but also improves repair capability. We anticipate that this threat will be further reduced as LLM costs decrease with ongoing advancements in model efficiency.

7 Related Work

APR research has entered the era of LLMs [59]. Researchers are devoted to designing novel repair strategies to maximize the repair potential of LLM. In the early LLM4APR studies, researchers adopted the fine-tuning paradigm [8, 13, 44] to allow LLMs to learn empirical knowledge used to guide bug repair from massive bug-fix history. For example, work such as VulRepair [5], CIRCLE [55], NTR [11] and VulAdvisor [56] follow the fine-tuning paradigm to learn defect repair knowledge. In addition, researchers have also utilized the prompt-learning paradigm [47, 48, 60] to stimulate the repair potential of LLMs by combining LLMs with traditional APR techniques to construct specific prompt templates. For example, AlphaRepair [48] and GAMMA [60] generate patches directly for fault locations by leveraging traditional template-based techniques to design specific repair template prompts. Later, with the rapid development of generative models, LLMs possess more powerful language comprehension and generation capabilities, and can receive longer context window sizes to process more complex information. Therefore, researchers proposed many ChatGPT-based APR tools. For example, ChatRepair [49] introduces the conversational repair paradigm to make LLMs work better guided by feedback information such as test execution; SRepair [50] leverages LLMs' programming language understanding to infer possible root causes to generate patches; ThinkRepair [54] enables LLMs to better implement reasoning to complete the repair by leveraging the chain of thought. Recently, LLM4APR research has begun to enter the era of agent [3], where researchers have allowed LLMs to take on different task roles, further expanding the repair capabilities of LLMs. For example, RepairAgent [2] enables the LLM-based agent to perform repair by invoking external tools, and FixAgent [24] realizes unified debugging by constructing the multi-agent collaboration paradigm. Unlike previous works, our work effectively combines repair ingredients with LLM that searches for internal ingredients to better analyze bug causes, and searches for external ingredients to better guide bug fixing.

8 Conclusion

In this work, we presented ReinFix, a framework that integrates repair ingredients to enhance the capabilities of LLMs for APR. ReinFix is designed as a two-phase approach including reasoning and solution. During the reasoning phase, we utilize dependency analysis for internal ingredients search to facilitate LLMs in root cause analysis. During the solution phase, we combine the buggy code and root cause as bug patterns for external ingredient search to guide the patch generation. The synergy of the two phases that build on the ReAct Agent improves the LLM's ability to generate patches. Our extensive experiments, including comparisons with state-of-the-art baselines and ablation studies, demonstrate that ReinFix consistently outperforms existing methods.

Acknowledgments

We thank all anonymous reviewers for their constructive feedback. This work was supported by the National Research Foundation, Singapore under the AI Singapore Programme (AISG Award No: AISG4-GC-2023-008-1B), the National Research Foundation, Singapore, the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN), and the National Research Foundation, Prime Minister's Office, Singapore under the Campus for Research Excellence and Technological Enterprise (CREATE) programme. This research was partially supported by a grant of API credits from the OpenAI Researcher Access Program. Kai Huang is supported by the Federal Ministry of Education and Research (BMBF) and a fellowship within the IFI programme of the German Academic Exchange Service (DAAD). Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the authors and do not reflect the views of the National Research Foundation, Singapore, the Cyber Security Agency of Singapore or OpenAI.

References

[1] DeepSeek AI. 2023. DeepSeek Coder: Let the Code Write Itself. https://github.com/deepseek-ai/DeepSeek-Coder

[2] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2024).

[3] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. *arXiv preprint arXiv:2506.18824* (2025).

[4] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair. *IEEE Transactions on Software Engineering (TSE)* 47, 9 (2019), 1943–1959.

[5] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based Automated Software Vulnerability Repair. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 935–947.

[6] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering (TSE)* 45, 01 (2019), 34–67.

[7] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 34, 5 (2025), 1–30.

[8] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An Empirical Study on Fine-Tuning Large Language Models of Code for Automated Program Repair. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1162–1174.

[9] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. 2024. Evolving Paradigms in Automated Program Repair: Taxonomy, Challenges, and Opportunities. *ACM Computing Surveys (CSUR)* 57, 2 (2024), 1–43.

[10] Kai Huang, Jian Zhang, Xinlei Bao, Xu Wang, and Yang Liu. 2025. Comprehensive Fine-Tuning Large Language Models of Code for Automated Program Repair. *IEEE Transactions on Software Engineering (TSE)* 51, 4 (2025), 904–928.

[11] Kai Huang, Jian Zhang, Xiangxin Meng, and Yang Liu. 2025. Template-Guided Program Repair in the Era of Large Language Models. In *47th International Conference on Software Engineering (ICSE)*. 367–379.

[12] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing is Fixing: Cross-Modal Reasoning with Multimodal LLMs for Visual Software Issue Fixing. *arXiv preprint arXiv:2506.16136* (2025).

[13] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1430–1442.

[14] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. 2023. KNOD: Domain Knowledge Distilled Tree Decoder for Automated Program Repair. In *45th International Conference on Software Engineering (ICSE)*. 1251–1263.

[15] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-Aware Neural Machine Translation for Automatic Program Repair. In *43rd International Conference on Software Engineering (ICSE)*. 1161–1173.

[16] joern.io. 2024. *Joern: The Bug Hunter's Workbench*. https://github.com/joernio/joern

[17] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *2014 International Symposium on Software Testing and Analysis (ISSTA)*. 437–440.

[18] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *35th International Conference on Software Engineering (ICSE)*. 802–811.

[19] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. 2024. Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *arXiv preprint arXiv:2403.01971* (2024).

[20] LangChain. 2024. Applications that can reason. Powered by LangChain. https://www.langchain.com/

[21] VI Lcvenshtcin. 1966. Binary coors capable or 'correcting deletions, insertions, and reversals. In *Soviet Physics-Doklady*, Vol. 10.

[22] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering (TSE)* 38, 01 (2012), 54–72.

[23] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated Program Repair. *Communications of the ACM (CACM)* 62, 12 (2019), 56–65.

[24] Cheryl Lee, Chunqiu Steven Xia, Jen-tse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R Lyu. 2024. A Unified Debugging Approach via LLM-Based Multi-Agent Synergy. *arXiv preprint arXiv:2404.17153* (2024).

[25] Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. DLFix: Context-based Code Transformation Learning for Automated Program Repair. In *42nd International Conference on Software Engineering (ICSE)*. 602–614.

[26] Yi Li, Shaohua Wang, and Tien N Nguyen. 2022. DEAR: A Novel Deep Learning-based Approach for Automated Program Repair. In *44th International Conference on Software Engineering (ICSE)*. 511–523.

[27] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large Language Model-Based Agents for Software Engineering: A Survey. *arXiv preprint arXiv:2409.02977* (2024).

[28] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting Template-based Automated Program Repair. In *28th International Symposium on Software Testing and Analysis (ISSTA)*. 31–42.

[29] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models using Ensemble for Program Repair. In *29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 101–114.

[30] Matias Martinez, Westley Weimer, and Martin Monperrus. 2014. Do the fix ingredients already exist? an empirical inquiry into the redundancy assumptions of program repair approaches. In *36th International Conference on Software Engineering (ICSE)*. 492–495.

[31] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2022. Improving Fault Localization and Program Repair with Deep Semantic Features and Transferred Knowledge. In *44th International Conference on Software Engineering (ICSE)*. 1169–1180.

[32] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, Xudong Liu, and Chunming Hu. 2023. Template-based Neural Program Repair. In *45th International Conference on Software Engineering (ICSE)*. 1456–1468.

[33] Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–24.

[34] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: Program Repair via Semantic Analysis. In *35th International Conference on Software Engineering (ICSE)*. 772–781.

[35] OpenAI. 2021. gpt-3.5-turbo-0125. https://platform.openai.com/docs/models#gpt-3-5-turbo

[36] OpenAI. 2021. gpt-4-0613. https://platform.openai.com/docs/models#gpt-4-turbo-and-gpt-4

[37] OpenAI. 2022. ChatGPT: Optimizing Language Models for Dialogue. https://openai.com/blog/chatgpt/

[38] OpenAI. 2023. gpt-4-1106-preview. https://platform.openai.com/docs/models#gpt-4-turbo-and-gpt-4

[39] OpenAI. 2023. gpt-4o-2024-05-13. https://platform.openai.com/docs/models#gpt-4-turbo-and-gpt-4

[40] OpenAI. 2024. Vector Embeddings. https://platform.openai.com/docs/guides/embeddings

[41] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 4 (2019), 1–29.

[42] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 146–158.

[43] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.

[44] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How Effective Are Neural Networks for Fixing Security Vulnerabilities. In *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 1282–1294.

[45] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.

[46] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. The Plastic Surgery Hypothesis in the Era of Large Language Models. In *38th International Conference on Automated Software Engineering (ASE)*. 522–534.

[47] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *45th International Conference on Software Engineering (ICSE)*. 1482–1494.

[48] Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 959–971.

[49] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *33rd International Symposium on Software Testing and Analysis (ISSTA)*. 819–831.

[50] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Haotian Zhang, and Yuqun Zhang. 2024. How Far Can We Go with Practical Function-Level Program Repair? *arXiv preprint arXiv:2404.12833* (2024).

[51] Deheng Yang, Kui Liu, Dongsun Kim, Anil Koyuncu, Kisub Kim, Haoye Tian, Yan Lei, Xiaoguang Mao, Jacques Klein, and Tegawendé F Bissyandé. 2021. Where were the repair ingredients for defects4j bugs? exploring the impact of repair ingredient retrieval on the performance of 24 program repair systems. *Empirical Software Engineering (EMSE)* 26 (2021), 1–33.

[52] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[53] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural Program Repair with Execution-based Backpropagation. In *44th International Conference on Software Engineering (ICSE)*. 1506–1518.

[54] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed Automated Program Repair. In *33rd International Symposium on Software Testing and Analysis (ISSTA)*. 1274–1286.

[55] Wei Yuan, Quanjun Zhang, Tieke He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: Continual Repair Across Programming Languages. In *31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 678–690.

[56] Jian Zhang, Chong Wang, Anran Li, Wenhan Wang, Tianlin Li, and Yang Liu. 2024. Vuladvisor: Natural language suggestion generation for software vulnerability repair. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1932–1944.

[57] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Yanjun Pu, and Xudong Liu. 2020. Learning to handle exceptions. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 29–41.

[58] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2023. A Survey of Learning-based Automated Program Repair. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 33, 2 (2023), 1–69.

[59] Quanjun Zhang, Chunrong Fang, Yang Xie, Yuxiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. 2024. A Systematic Literature Review on Large Language Models for Automated Program Repair. *arXiv preprint arXiv:2405.01466* (2024).

[60] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting Template-Based Automated Program Repair via Mask Prediction. In *38th International Conference on Automated Software Engineering (ASE)*. 535–547.

[61] Qihao Zhu, Zeyu Sun, Yuanan Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A Syntax-Guided Edit Decoder for Neural Program Repair. In *29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 341–353.

[62] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-aware Neural Program Repair. In *45th International Conference on Software Engineering (ICSE)*. 1443–1455.